

Diagonal-Aware SpMV and SpMSpM Kernels for GPU-Accelerated Hamiltonian Simulation

Yuchao Su

Department of Computer Science
NC State University
Raleigh, USA
ysu34@ncsu.edu

Frank Mueller

Department of Computer Science
NC State University
Raleigh, USA
fmuelle@ncsu.edu

Abstract—Hamiltonian simulation plays a key role in quantum computing as such quantum algorithms, after classical simulation, are seen as a means of validating quantum devices before they are deployed. In Hamiltonian simulation, state-vector evolution and explicit operator construction are two dominant computational components. State-vector simulation repeatedly applies a Hamiltonian, or a low-order polynomial in the Hamiltonian, to a quantum state, making sparse matrix-vector multiplication (SpMV) the dominant kernel. Explicit operator construction forms derived sparse operators, making sparse matrix-matrix multiplication (SpMSpM) the dominant kernel.

Hamiltonians arising in quantum simulation are highly structured and contain only a small set of active diagonal offsets. However, current GPU sparse kernels do not fully exploit this diagonal structure. To address these challenges, we propose two diagonal-aware GPU kernels: a reconstructed diagonal-tile SpMV kernel that maps local diagonal computation to tensor-core matrix multiplication, and a guided planner for the diagonal-interaction SpMSpM kernel that shifts output discovery to a host-side planning stage before tiled GPU execution. We evaluate both kernels on representative diagonal-sparse Hamiltonians from HamLib using an NVIDIA H100 GPU.

I. INTRODUCTION

Hamiltonian simulation is an important component of quantum computing, as it enables the study of physical quantum systems and serves as a classical tool for validating quantum algorithms before they run on quantum devices [4], [8], [11]. Two explicit computational modes dominate this workload. In state-vector simulation (e.g., QuEST [8] and many others), the simulator repeatedly applies a Hamiltonian or a polynomial in the Hamiltonian to a quantum state, making sparse matrix-vector multiplication (SpMV) the core kernel. In explicit operator construction, the simulator builds derived operators such as truncated-Taylor propagator approximations or reusable sparse operators in packages such as QuTiP, making sparse matrix-matrix multiplication (SpMSpM) the core kernel [1], [7]. Many Hamiltonians are sparse with structured diagonals, but this structure remains underexploited by current GPU sparse kernels and libraries [2], [10], [12].

Existing GPU sparse kernels mostly target general sparse formats and only partially exploit diagonal structure. For SpMV, diagonal-aware variants exist, but diagonals can still have highly uneven lengths, so a direct mapping to GPU work units leads to load imbalance and poor utilization [3], [10]. For

SpMSpM, GPU SpGEMM is already difficult because output discovery and accumulation are irregular. Furthermore, in diagonal storage, cross-diagonal interactions still create scattered updates, and atomic accumulation becomes increasingly severe as matrix dimension grows [5], [13], [14]. As a result, general-purpose sparse kernels fail to capture the regularity that these Hamiltonian matrices actually expose.

To address this gap, we propose two diagonal-aware GPU kernels tailored to the two dominant modes of Hamiltonian simulation: a diagonal-aware SpMV kernel for applying a Hamiltonian to a state vector, and a diagonal-aware SpMSpM kernel for multiplying two diagonal-structured sparse operators. The SpMV kernel reconstructs local diagonal regions into tensor-core-friendly tiles, converting the product stage of single-vector SpMV into dense matrix multiplication followed by a local row-wise reduction. The SpMSpM kernel organizes diagonal-pair products by their output offsets and uses a predicated execution plan followed by GPU tile execution to reduce irregular accumulation.

II. BACKGROUND

A. Hamiltonian Simulation

Throughout this paper, we use units with $\hbar = 1$. Hamiltonian dynamics is governed by

$$\psi(t + \Delta t) = e^{-iH\Delta t}\psi(t), \quad (1)$$

where $H \in \mathbb{C}^{N \times N}$ is the Hamiltonian and $\psi \in \mathbb{C}^N$ is the quantum state. A classical simulator can approximate the action of $e^{-iH\Delta t}$ in two different ways, which lead directly to two different sparse kernels.

1) *Single-State Propagation*: In this mode the simulator never forms the propagator explicitly, but approximates the action of $e^{-iH\Delta t}$ on a single state vector, one time step at a time. A standard approach is the Krylov subspace method, which builds its subspace by repeatedly applying H to the current state [6]. The propagation thus reduces to a sequence of sparse matrix-dense vector products, with no derived operator materialized, making SpMV the key kernel.

2) *Explicit Operator Construction*: In this mode the simulator explicitly constructs a derived sparse operator. A common

example is a truncated Taylor approximation to the propagator [1],

$$e^{-iH\Delta t} \approx \sum_{k=0}^K \frac{(-iH\Delta t)^k}{k!}, \quad (2)$$

forming higher-order Hamiltonian terms explicitly, which shifts the cost to sparse matrix–matrix multiplication. This mode is useful when an explicit operator is reused across multiple applications, time steps, or analysis tasks, as in packages such as QuTiP [7].

Thus, SpMV and SpMSpM arise not as generic sparse primitives, but as the two basic computational modes of classical Hamiltonian simulation.

B. Diagonal Structure

For Hamiltonians in quantum simulation, the nonzeros are concentrated on a small set of diagonal offsets [9]. Compact diagonal formats exploit this by storing only active offsets and their associated values. This view is attractive because it exposes the algebraic regularity of the workload directly: Nonzeros are organized by offset, which is shown in Fig. 1 rather than by individual row-column coordinates, and interactions between diagonals are more structured than in general sparse matrices.

However, diagonal structure does not completely remove irregularity. Although most interior diagonal regions can be reconstructed into dense tiles, boundary regions still require padding because shifted diagonals fall outside the matrix range. In addition, SpMSpM introduces cross-diagonal interactions, where each input diagonal pair contributes to an output diagonal determined by the sum of their offsets. These properties create two GPU challenges: mapping diagonal SpMV to efficient execution while handling boundary padding, and organizing structured but nonuniform diagonal-pair interactions in SpMSpM. The next section builds on this observation and presents kernel designs that treat diagonals as the primary unit of computation.

III. DIAGONAL-AWARE SPARSE KERNELS

This work designs diagonal-aware GPU kernels for the two sparse computational paths in Hamiltonian simulation.

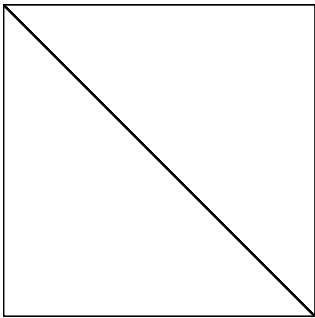


Fig. 1: Sparsity pattern of the *c2h-16* Hamiltonian from Hamlib ($2^{16} \times 2^{16}$ matrix). Non-zero entries are restricted to diagonals $d \in \{-64, -32, -16, -8, -4, -2, -1, 1, 2, 4, 8, 16, 32, 64\}$.

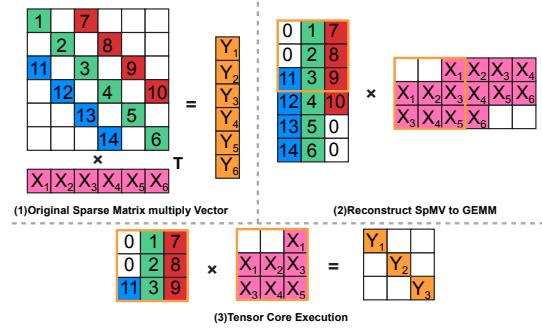


Fig. 2: Example of the SpMV kernel. (1) original sparse matrix-vector multiplication in dense matrix form, (2) reconstruction of SpMV as a GEMM-compatible operation, and (3) Tensor Core execution on the reconstructed tiles.

The SpMV kernel targets state-vector evolution, where the Hamiltonian is applied to a quantum state. The SpMSpM kernel targets operator construction, where powers of the Hamiltonian are generated for Taylor-expansion-based time evolution. Both kernels treat diagonals as the primary unit of computation rather than mapping the Hamiltonian to generic sparse format.

A. Diagonal-Aware SpMV Kernel

The SpMV kernel targets Hamiltonian applications using the diagonal representation of H . Let D denote the set of active diagonal offsets. For each $d \in D$, the compact diagonal array H_d has length $N - |d|$ and stores

$$H_d[j] = H_{j+\max(0,-d), j+\max(0,d)}, \quad 0 \leq j < N - |d|. \quad (3)$$

Using this local diagonal index j , the contribution of diagonal d to the state vector is

$$y_{j+\max(0,-d)} += H_d[j] \cdot x_{j+\max(0,d)}. \quad (4)$$

This formulation preserves the compact DIA storage layout while making the shifted access pattern of each diagonal explicit.

As illustrated in Fig. 2, we reconstruct the diagonal format SpMV into a tensor-core-friendly matrix multiplication. For a local tile, the diagonal values that fall into the tile are materialized into a dense matrix operand A_{tile} according to their original local row and column positions. The corresponding input-vector values are arranged into a reconstructed matrix operand B_{tile} . The tensor core then computes

$$P_{\text{tile}} = A_{\text{tile}} B_{\text{tile}}. \quad (5)$$

The valid entries of the temporary product P_{tile} encode the products between Hamiltonian entries and their corresponding vector elements. The SpMV output for the tile is then obtained by a local reduction over the valid entries:

$$y_{\text{tile}}[r] = \sum_c P_{\text{tile}}[r, c]. \quad (6)$$

This reconstruction packs multiple diagonal contributions into the same tensor-core tile. Instead of processing each diagonal independently, the values from different active diagonals are placed into one local matrix operand and multiplied with the reconstructed vector operand together. As a result, the tensor-core computation generates the products for multiple diagonals in one matrix-multiply operation, and the following local reduction combines the products that contribute to the same output entry.

The reconstructed matrix is dense for most of the computation. Zero padding appears only in the first and last several rows, where shifted diagonals exceed the matrix boundary and therefore do not have valid entries. For the interior rows, all active diagonals contribute valid values, so the reconstructed tile is **dense**. This property allows the main SpMV computation to use dense tensor-core execution, while the boundary rows introduce only limited padding overhead. The strategy also improves workload regularity. Although different diagonals have different valid lengths in the original diagonal representation, the reconstructed tensor-core tiles have a uniform shape. Boundary zeros are inserted only where shifted diagonals exceed the matrix boundary, while interior rows remain dense. As a result, most of the computation is mapped to dense tiles with the same tensor-core dimensions, which balances the workload across GPU execution units and reduces irregular control flow.

Overall, the SpMV kernel keeps the Hamiltonian in compact diagonal form in memory and reconstructs tensor-core operands only at execution time. The product stage is mapped to dense tensor-core matrix multiplication. This design exploits the regular diagonal structure of Hamiltonian matrices while converting most of the SpMV computation into dense tensor-core operations.

B. Diagonal-Aware SpMSPM Kernel

The SpMSPM kernel accelerates Taylor-expansion-based operator construction (Equation 2). To approximate the time-

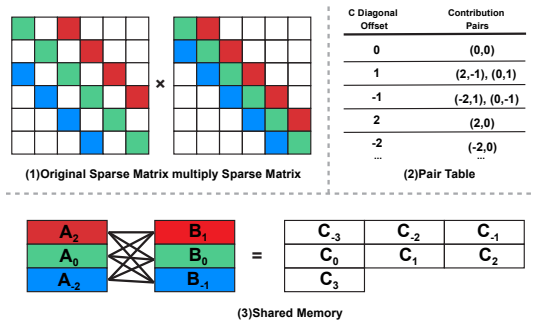


Fig. 3: Example of the SpMSPM kernel. (1) original sparse matrix-matrix multiplication in dense matrix form, (2) diagonal-pair relationship among A , B , and C , where each output diagonal is generated by pairs (d_A, d_B) satisfying $d_C = d_A + d_B$, and (3) kernel-level view showing tiled diagonal loading into shared memory and accumulation into output diagonals.

evolution operator, the simulator constructs Hamiltonian powers up to order K through repeated sparse matrix-matrix multiplication.

For diagonal-sparse matrices, SpMSPM has a structured form. An input diagonal from A with offset d_A and an input diagonal from B with offset d_B contribute only to the output diagonal with offset $d_C = d_A + d_B$. For each valid global row index i , the contribution is

$$C_{i, i+d_A+d_B} += A_{i, i+d_A} \cdot B_{i+d_A, i+d_A+d_B}. \quad (7)$$

For implementation, we use the local index j of the output diagonal d_C . The same interaction can be written as

$$C_{d_C}[j] += A_{d_A}[j + \Delta_A] \cdot B_{d_B}[j + \Delta_B], \quad (8)$$

where

$$\Delta_A = \max(0, -d_C) - \max(0, -d_A), \quad (9)$$

$$\Delta_B = \max(0, -d_C) + d_A - \max(0, -d_B). \quad (10)$$

The interaction is valid only when both referenced input indices lie within the bounds of their compact diagonal arrays.

As illustrated in Fig. 3, the diagonal interaction pattern can be determined directly from diagonal offsets. This pair table removes the need to discover output locations through irregular sparse traversal during GPU execution. Based on this observation, our design uses a host-side schedule planner to enumerate valid diagonal interactions before launching the GPU kernel. Each planned operation records the input diagonal offsets d_A and d_B , the output offset d_C , and the shifts Δ_A and Δ_B required to align the compact diagonal indices. Interactions that map to the same output diagonal are grouped together so that their contributions can be accumulated in a regular output tile.

The GPU kernel then processes the planned interactions tile by tile. For each output diagonal tile, the kernel loads the required A - and B -diagonal segments into shared memory, initializes the corresponding C tile, and performs the planned pairwise multiply-accumulate operations. After all contributing pairs for the tile are processed, the kernel writes the completed output diagonal tile back to global memory. This execution model replaces irregular output discovery and fine-grained atomic [5] with a planned diagonal-pair schedule and shared-memory tile accumulation.

IV. EVALUATION

We evaluate the proposed kernels on representative Hamiltonians from HamLib [12]. We focus on the c2h and B2 families from HamLib because they span very small to very large numbers of active diagonals, thereby stressing both diagonal-length imbalance in SpMV and interaction density in SpMSPM. All experiments target an NVIDIA H100 GPU. We compare SpMV against cuSPARSE [10]. For SpMSPM, we use cuSPARSE and the prior diagonal-format GPU method of Haque et al. [5], referred to “Haque” in the following.

Algorithm 1: Tiled Diagonal-Pair SpMSPM

Input: Input diagonals A, B ; pair list $\mathcal{P}$; tile size T
Output: Output diagonals C

```

1 foreach thread block  $b$  in parallel do
2   Load block metadata: input diagonals, output
   diagonals, and pair list  $\mathcal{P}^b$ ;
3   foreach index tile  $t$  do
4     Load required  $A$  segments into shared memory;
5     Load required  $B$  segments into shared memory;
6     Clear shared output tile  $C$ ;
7     Synchronize threads;
8     foreach pair entry  $(a, b, c, \Delta_A, \Delta_B) \in \mathcal{P}^b$  do
9       foreach local index  $j$  in tile  $t$  in parallel do
10         $C_c[j] += A_a[j + \Delta_A] \times B_b[j + \Delta_B]$ ;
11     Synchronize threads;
12     Write diagonal tile  $C$  back to global memory;
```

A. Experimental Setup

Table I summarizes the four HamLib matrices used in our study. The *c2h* instances contain only 8 and 14 active diagonals, while the *B2* instances contain 83 and 1325 active diagonals. For SpMV, we report end-to-end kernel time for applying H to a dense state vector after converting the Hamiltonian into diagonal storage. For SpMSPM, we evaluate multiplying each Hamiltonian by itself, which captures the operator-building workload that appears in truncated-Taylor and related propagation methods.

TABLE I: HamLib matrices used in our evaluation.

Benchmark	Qubits	Name	Dim	Sparsity	Active Diagonals
c2h	10	<i>c2h-10</i>	1,024	99.70%	8
c2h	16	<i>c2h-16</i>	65,536	99.99%	14
B2	10	<i>b2-10</i>	1,024	99.34%	83
B2	14	<i>b2-14</i>	16,384	99.84%	1325

B. SpMV Performance

Figure 4 shows that the proposed packed diagonal-tile SpMV kernel consistently outperforms cuSPARSE on all four matrices. The speedup ranges from $3.9\times$ to $5.6\times$, with an average gain of $4.7\times$. The best result appears on the larger *c2h* instance *c2h-16*, where the long but relatively few diagonals map cleanly to balanced tiles. The lowest gain appears on *b2-10*, but the kernel still achieves $3.9\times$ speedup. Overall, the results show that local reconstruction enables efficient tensor-core SpMV by converting most diagonal computation into dense tile operations with only limited boundary padding.

C. SpMSPM Performance

Figure 5 shows that our planned diagonal-interaction kernel delivers the strongest gains in the evaluation. It reaches $5.8\times$ – $7.2\times$ speedup over cuSPARSE, with an average gain of $6.4\times$ across the four matrices. It also remains consistently ahead

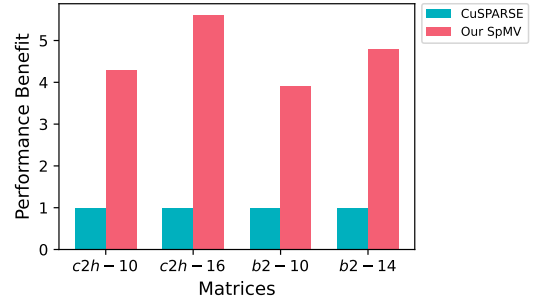


Fig. 4: Normalized SpMV speedup relative to cuSPARSE on the four HamLib matrices in Table I.

of Haque, with an additional $1.62\times$ – $1.99\times$ improvement and an average gain of $1.81\times$. The largest relative gain appears on *c2h-16*, while the high-diagonal-count B2 cases also show clear improvement. These results are consistent with our design goal: By moving output discovery to a host-side planner and executing precomputed diagonal interactions on the GPU, the kernel reduces irregular accumulation and avoids much of the atomic-update overhead in Haque which limits generic sparse multiplication.

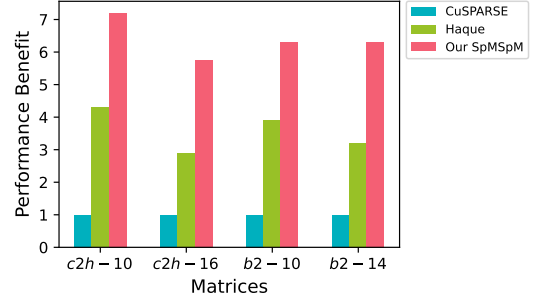


Fig. 5: Normalized SpMSPM speedup relative to cuSPARSE and Haque on the four HamLib matrices in Table I.

V. CONCLUSION

This paper presents diagonal-aware GPU kernels for the two dominant sparse workloads in classical Hamiltonian simulation: sparse matrix-vector multiplication (SpMV) for state-vector simulation and sparse matrix-matrix multiplication (SpMSPM) for explicit operator construction. On representative HamLib matrices on an NVIDIA H100 GPU, the proposed kernels deliver $3.9\times$ – $5.6\times$ speedup for SpMV over cuSPARSE and $5.8\times$ – $7.2\times$ speedup for SpMSPM over cuSPARSE, while also consistently outperforming Haque for SpMSPM. These results show that diagonal-aware scheduling is a practical and effective basis for accelerating Hamiltonian simulation on modern GPUs.

ACKNOWLEDGMENTS

This work was supported in part by NSF CISE-2217020, CISE-2316201, OMA-2120757, PHY-1818914 and PHY-2325080.

REFERENCES

- [1] D. W. Berry, A. M. Childs, R. Cleve, R. Kothari, and R. D. Somma, “Simulating hamiltonian dynamics with a truncated taylor series,” *Physical review letters*, vol. 114, no. 9, p. 090502, 2015.
- [2] S. Chundury, J. Li, I.-S. Suh, and F. Mueller, “Diaq: Efficient state-vector quantum simulation,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.01250>
- [3] J. Gao, Y. Xia, R. Yin, and G. He, “Adaptive diagonal sparse matrix-vector multiplication on gpu,” *Journal of Parallel and Distributed Computing*, vol. 157, pp. 287–302, 2021. [Online]. Available: <https://doi.org/10.1016/j.jpdc.2021.07.007>
- [4] T. Häner, D. S. Steiger, K. M. Svore, and M. Troyer, “High performance emulation of quantum circuits,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC)*, 2016, pp. 1–10. [Online]. Available: <https://doi.org/10.1109/SC.2016.73>
- [5] S. A. Haque, M. T. Parvez, and S. Hossain, “Gpu algorithms for structured sparse matrix multiplication with diagonal storage schemes,” *Algorithms*, vol. 17, no. 1, p. 31, 2024. [Online]. Available: <https://doi.org/10.3390/a17010031>
- [6] M. Hochbruck and C. Lubich, “On krylov subspace approximations to the matrix exponential operator,” *SIAM Journal on Numerical Analysis*, vol. 34, no. 5, pp. 1911–1925, 1997.
- [7] J. R. Johansson, P. D. Nation, and F. Nori, “Qutip 2: A python framework for the dynamics of open quantum systems,” *Computer Physics Communications*, vol. 184, no. 4, pp. 1234–1240, 2013. [Online]. Available: <https://doi.org/10.1016/j.cpc.2012.11.019>
- [8] T. Jones, A. Brown, I. Bush, and S. C. Benjamin, “Quest and high performance simulation of quantum computers,” *Scientific Reports*, vol. 9, no. 1, Jul. 2019. [Online]. Available: <http://dx.doi.org/10.1038/s41598-019-47174-9>
- [9] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010. [Online]. Available: <https://doi.org/10.1017/CBO9780511976667>
- [10] *cuSPARSE Library*, NVIDIA Corporation. [Online]. Available: <https://docs.nvidia.com/cuda/cusparses/index.html>
- [11] J. Preskill, “Quantum computing in the nisq era and beyond,” *Quantum*, vol. 2, p. 79, 2018. [Online]. Available: <https://doi.org/10.22331/q-2018-08-06-79>
- [12] N. P. Sawaya, D. Marti-Dafcik, Y. Ho, D. P. Tabor, D. E. B. Neira, A. B. Magann, S. Premaratne, P. Dubey, A. Matsuura, N. Bishop, W. A. d. Jong, S. Benjamin, O. Parekh, N. Tubman, K. Klymko, and D. Camps, “HamLib: A library of Hamiltonians for benchmarking quantum algorithms and hardware,” *Quantum*, vol. 8, p. 1559, Dec. 2024. [Online]. Available: <https://doi.org/10.22331/q-2024-12-11-1559>
- [13] Z. Xie, G. Tan, W. Liu, and N. Sun, “IA-SpGEMM: An input-aware auto-tuning framework for parallel sparse matrix-matrix multiplication,” in *Proceedings of the ACM International Conference on Supercomputing*, 2019, pp. 94–105. [Online]. Available: <https://doi.org/10.1145/3330345.3330354>
- [14] D. Yao, S. Zhao, T. Liu, G. Wu, and H. Jin, “ApSpGEMM: Accelerating large-scale spgemm with heterogeneous collaboration and adaptive panel,” *ACM Transactions on Architecture and Code Optimization*, vol. 22, no. 1, pp. 1–23, 2025. [Online]. Available: <https://doi.org/10.1145/3703352>